

The run-up to Wednesday

Monday–Tuesday: build and fix. Tuesday evening: test. Wednesday: deploy.

1. The prompt is 34,278 characters ← biggest win

Every message carries 34,278 characters, about 8,500 tokens. That is the slowness, and it is paid on every "hello".

Part	Size	Loads now	Should load
AMI_SCRIPT — her character, 17 sections	15,757	always	always, but trim
~20 context blocks	~12,000	whenever data exists	when the question needs it
Recent memory	~2,200	always	always
Briefing	9,253	gated 3 Oct	news questions only

The twenty: health, conditions, blood sugar, cholesterol, measurements, training, prices, tailor sizes, garments, BP, family, taught facts, travel, subscriptions, medication, goals, birthdays. Each gated on *having data*, not relevance. He has data for all of them.

Target: under 15,000 for an ordinary message. Risk: the function that broke twice on 3 Oct. Backup, one pass, revert fast.

2. One door to the model

Gemini is called four different ways: `client.models.generate_content`, `genai.GenerativeModel`, via `_ask_gemini`, and inside `synthesize_response` in a form grep could not find. That is why a fix lands in one place and not another — it cost three wrong theories on 3 Oct.

Every call goes through one function. Then model, retries, fallback, cost and errors are decided once.

3. Projects, properly ← the new one

He can create tasks but not projects, so a plan she writes in chat dies there. `ventures` already has what is needed: `type` (venture/project), `parent_id`, milestones, risks, progress, next_action. The data model is fine; the flow is missing.

One flow, two doors — chat or the Projects screen: 1. He describes it ("a site to build resumes and apply for jobs") 2. She gives her honest read and the phases — *she already does this well* 3. He says "do it" 4. She creates the project and **proposes** tasks 5. He ticks what is real, edits wording, drops the rest 6. Nothing lands unticked

Also: - A project dropdown when creating or editing any task - From a project page: add more tasks, see what is done and stuck - In chat: "that task is for the Freetown building" attaches it

Principle: she proposes, he disposes. Every problem this week came from something created or claimed without him seeing it.

4. The Tasks board — one careful pass

- The grouping modal: two tabs, "What belongs together" / "Tidy the wording". Backend is **done and working** (`/api/tasks/group`); the modal broke the build on 3 Oct and was reverted.
- Rename "AI Organize" → **"Sort out the board"**
- Add task always visible; Settings in the menu. Right now the thing he does most often and the thing he never does take the same two taps.
- Show more across the whole board, not per column
- Filter by source — from Ami, from notes, from him
- Origin follows into todos: "from Ami-Tasks", not just "from task"

- **Delete** `TasksPage.jsx` — 235 dead lines, imported nowhere, and both of us kept editing it instead of `TasksKanban.jsx`

5. Only calls that count

- Audit all call sites
- Retry: once, only on an explicit overload, never on a timeout
- More instant answers. Done: clock, currency, counts, prices, water, API spend. Candidates: "what's on today" (maybe — it is a small briefing), "when is X playing", "what's due this week"

6. Dead code

- `if False:` around the briefing responder (~3733)
- Old learning tables: 5 with code, 0 rows
- `/api/report` superseded by `/api/report2`
- `ami_identity.py`, `kb_loader.py` — decide
- `app.py.before_*`, `*.bak-*`, and my patch scripts

7. Small but real

- Duplicate fixture on the strip — one line per match, not per team
- Chat opens mid-scroll when returning from another tab
- Notifications — needs a proper conversation, not a toggle

8. Wednesday: deploy

- [] Restore endpoint back temporarily
- [] Database up
- [] Exercise media (71 files, 9.8MB) and medical documents (4)
- [] **Remove the restore endpoint**
- [] Frontend deployed — only the API is up
- [] Backups off the server
- [] Prove the volume survives a redeploy

9. Then the tutorial

Deployment chapter written from doing it, plus: how Ami was trained; engines vs modules properly; how API calls work, with diagrams; emojis and markers that cannot be faked; the retry that cost money invisibly; and **the three hours lost to a billing hold that reported itself as a Gemini outage.**

From the weekend

- [] Whatever his testing turns up